

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

КУРСОВОЙ ПРОЕКТ

по курсу
«Разработка веб-приложений»

ТЕМА

«Разработка веб-приложения для частной клиники с системой записи на приём, медицинскими картами и личными кабинетами»

Выполнил: Амплеенков Даниил Олегович

Группа: 241-327

Проверил: Кружалов Алексей Сергеевич

Москва, 2026

**«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)**

УТВЕРЖДАЮ
заведующая кафедрой
«Инфокогнитивные технологии»

_____ / Е. А. Пухова /

«____» _____ 2026 г.

ЗАДАНИЕ

на выполнение курсовой работы (проекта)

Амплеенкову Даниилу Олеговичу,

обучающемуся группы 241-327,

направления подготовки 09.03.01 «Информатика и вычислительная техника»

по дисциплине «Разработка веб-приложений»

на тему «Разработка веб-приложения для частной клиники с системой записи на приём, медицинскими картами и личными кабинетами»

1. Исходные данные к работе (проекту): информационные ресурсы в сети интернет, научные публикации в открытой печати.

2. Содержание задания по курсовой работе (проекту) – перечень вопросов, подлежащих разработке:

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Раздел 1. Анализ предметной области			
Задача 1.1. Обзор существующих программных продуктов по теме работы	10	3.04.2026	
Задача 1.2. Анализ программных инструментов разработки веб-приложений	7	10.04.2026	
Задача 1.3. Формулировка цели и задач работы	8	18.04.2026	
Раздел 2. Проектирование веб-приложения			
Задача 2.1. Анализ целевой аудитории	5	25.04.2026	
Задача 2.2. Описание функциональности приложения (диаграмма вариантов использования, user story)	7	02.05.2026	

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Задача 2.3. Проектирование модели данных (ER-диаграмма, логическая и физическая схемы БД)	8	9.05.2026	
Задача 2.4. Разработка макетов страниц (Wireframe)	5	16.05.2026	
Раздел 3. Разработка веб-приложения			
Задача 3.1. Разработка базовой структуры приложения и вёрстка шаблонов страниц	8	23.05.2026	
Задача 3.2. Реализация аутентификации пользователей	7	30.05.2026	
Задача 3.3. Реализация CRUD-интерфейса для пользователей, записей на приём и медицинских карт с поддержкой загрузки и выгрузки файлов	7	02.06.2026	
Задача 3.4. Реализация системы управления расписанием и онлайн-записью на приём	12	04.06.2026	
Задача 3.5. Разработка личного кабинета с просмотром медкарты и истории посещений	6	06.06.2026	
Раздел 4. Оформление итогов работы			
Задача 4.1. Создание Git-репозитория с кодом проекта	3	7.06.2026	
Задача 4.2. Деплой приложения на хостинг	4	10.06.2026	
Задача 4.3. Оформление отчёта о проделанной работе	3	13.06.2026	

Руководитель курсовой работы (проекта):
старший преподаватель кафедры «Инфокогнитивные технологии»

«____» _____
2026 г.

(подпись)

А. С. Кружалов

Дата выдачи задания

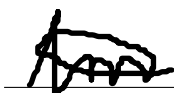
«30» марта 2026 г.

Дата сдачи выполненной работы

«15» июня 2026 г.

Задание принял к исполнению

«03» апреля 2026 г.



(подпись)

Д. О. Амплеевков

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	6
1.1 Обзор существующих программных продуктов по теме работы	6
1.2 Анализ программных инструментов разработки веб-приложений	9
1.3 Формулировка цели и задач работы	11
ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ	13
2.1 Анализ целевой аудитории	13
2.2 Описание функциональности приложения	14
2.3 Проектирование модели данных	16
2.4 Разработка макетов страниц	20
2.5 Контейнеризация и развертывание с использованием Docker .	24
ГЛАВА 3. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ ВЕБ-ПРИЛОЖЕНИЯ	27
3.1 Структура программного решения	27
3.2 Модели данных и ORM-маппинг	28
3.3 Логика маршрутизации и бизнес-логика	31
3.4 Интерфейс пользователя и экраны приложения	34
ЗАКЛЮЧЕНИЕ	38

ВВЕДЕНИЕ

Актуальность темы. Частные медицинские клиники ежедневно обрабатывают записи пациентов, расписание врачей, результаты приёмов, медицинские документы и обращения администраторов. При использовании бумажных журналов, отдельных таблиц или несвязанных программных сервисов возрастает вероятность ошибок при записи на приём, усложняется контроль загрузки врачей и затрудняется быстрый доступ к медицинской информации пациента. Поэтому актуальной является разработка веб-приложения, которое объединяет онлайн-запись, электронные медицинские карты и личные кабинеты пользователей в единой информационной системе.

Степень разработанности проблемы. На рынке существуют готовые медицинские информационные системы и пациентские сервисы, например Инфоклиника.RU, Medesk, ONDOC и Cliniccards. Они предоставляют инструменты для записи пациентов, ведения медицинских карт, работы с расписанием и коммуникации с пациентами [1–4]. Однако такие решения являются закрытыми коммерческими продуктами, рассчитанными на внедрение в конкретной клинике и не предназначенными для изучения внутренней архитектуры в рамках учебного проекта. Это обосновывает необходимость разработки веб-приложения, в котором основные процессы частной клиники представлены в упрощённом, но целостном виде.

Объект исследования - процессы цифрового взаимодействия частной клиники с пациентами, врачами и администраторами.

Предмет исследования - программные средства, модели данных и пользовательские сценарии, применяемые при разработке веб-приложения для онлайн-записи, ведения медицинских карт и организации личных кабинетов.

Целью данной работы является проектирование веб-приложения для частной клиники, обеспечивающего онлайн-запись пациентов на приём, хранение медицинской информации, работу с расписанием врачей и разграничение доступа между пациентом, врачом и администратором.

Для достижения поставленной цели необходимо решить следующие задачи: выполнить обзор существующих программных продуктов, проанализировать инструменты веб-разработки и существующие технологические стеки, выбрать технологию, сформулировать требования к системе, спроектировать целевую аудиторию, функциональность, модель данных и макеты страниц.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор существующих программных продуктов по теме работы

Предметная область проекта связана с автоматизацией работы частной медицинской клиники. К основным процессам такой организации относятся регистрация пациентов, запись на приём, ведение расписания врачей, оформление результатов посещения, хранение медицинских данных и предоставление пациенту доступа к информации о проведённом лечении.

В традиционном варианте часть этих процессов выполняется вручную или с использованием разрозненных программных средств. Администратор принимает звонки и фиксирует запись, врач ведёт медицинские записи в отдельной системе или документе, пациент узнаёт информацию о визитах через администратора. Такая схема имеет несколько недостатков: дублирование данных, риск ошибки при выборе времени приёма, сложность поиска медицинской информации, отсутствие единой истории посещений и повышенная нагрузка на сотрудников.

Разработка веб-приложения позволяет объединить основные процессы в единой информационной системе. Пациент получает возможность самостоятельно записываться на приём и просматривать сведения о визитах через личный кабинет, врач - работать с медицинскими картами и результатами приёмов, а администратор - управлять пользователями, расписанием, услугами и записями. В результате повышается прозрачность работы клиники, сокращается количество ручных операций и улучшается качество обслуживания пациентов.

Для определения функциональных требований были рассмотрены существующие программные продукты, предназначенные для автоматизации медицинских организаций и цифрового взаимодействия с пациентами. В качестве аналогов выбраны Инфоклиника.RU, Medesk, ONDOC и Cliniccards. Эти решения относятся к одной предметной области, но отличаются назначением: одни ориентированы на комплексное управление клиникой, другие - на личный кабинет пациента или на клиники определённого профиля. Поэтому сравнение выполнено в формате «критерий - платформа», что позволяет увидеть, какие функции необходимо включить в разрабатываемое приложение.

Таблица 1.1 — Анализ существующих платформ по функциональным критериям

Критерий	Инфоклиника.RU	Medesk	ONDOC	Cliniccards
Назначение платформы	Медицинская информационная система и электронная регистратура для клиники [1].	Облачная медицинская информационная система и CRM для управления клиникой [2].	Пациентский сервис и личный кабинет, ориентированный на хранение медицинских данных [3].	CRM-система для стоматологических клиник и управления пациентской базой [4].
Онлайн-запись на приём	Поддерживается через электронную регистратуру: пациент выбирает специалиста и время.	Поддерживается как часть расписания, CRM и карточки пациента.	Поддерживается со стороны пациента и зависит от подключения клиники к сервису.	Используется вместе с расписанием, но сценарии ориентированы преимущественно на стоматологию.
Личный кабинет пациента	Позволяет просматривать сведения о посещениях, результатах и рекомендациях.	Реализован частично: основной акцент сделан на внутренних процессах клиники.	Является ключевой функцией: пациент получает доступ к медицинской информации и взаимодействию с клиникой.	Имеются элементы работы с карточкой пациента, но личный кабинет пациента не является основной функцией продукта.
Электронная медицинская карта	Поддерживается как часть комплексной медицинской системы.	Поддерживает протоколы осмотров, историю записей и результаты исследований.	Даёт пациенту доступ к накопленным медицинским сведениям и документам.	Поддерживается с учётом стоматологической специфики, включая историю лечения пациента.
Управление расписанием врачей	Расписание связано с электронной регистратурой и используется для записи на приём.	Расписание применяется для планирования загрузки врачей и управления визитами.	Возможности зависят от интеграции с информационной системой клиники.	Расписание используется для планирования приёмов и ведения пациентской базы.
Ролевая модель доступа	Предусмотрены разные уровни доступа для сотрудников клиники, врача и пациента.	Предусмотрены роли сотрудников и механизмы разграничения процессов.	Основной пользователь - пациент; внутренняя ролевая модель зависит от интеграции с клиникой.	Предусмотрены роли сотрудников клиники и доступ к рабочим разделам CRM.
Работа с медицинскими документами	Возможен просмотр результатов анализов, заключений и рекомендаций.	Поддерживаются лабораторные результаты, медицинские документы и данные приёмов.	Поддерживается хранение медицинских данных и документов пациента.	Возможна работа с данными пациента и файлами в рамках CRM-системы.
Администрирование клиники	Включает управление пациентами, расписанием, приёмами и медицинскими процессами.	Включает CRM-функции, управление клиникой, пациентской базой и коммуникациями.	Ограничено, так как продукт ориентирован прежде всего на пациентский сценарий.	Включает управление пациентами, расписанием и рабочими процессами стоматологической клиники.
Специализация решения	Подходит для клиник разного профиля, где нужна электронная регистратура и МИС.	Подходит для частных клиник, которым требуется единая система управления процессами.	Подходит для клиник, которые хотят предоставить пациентам цифровой доступ к медицинской информации.	Подходит прежде всего для стоматологических клиник и профильных медицинских центров.

Продолжение таблицы 1.1

Критерий	Инфоклиника.RU	Medesk	ONDOC	Cliniccards
Открытость архитектуры для анализа	Низкая: коммерческая закрытая система.	Низкая: готовый облачный коммерческий сервис.	Низкая: внутренняя архитектура и логика интеграций не раскрываются.	Низкая: закрытая CRM-система с готовыми сценариями работы.

Анализ существующих платформ показывает, что цифровые сервисы для клиник обычно объединяют несколько обязательных направлений: онлайн-запись, управление расписанием, ведение электронной медицинской карты, работу с медицинскими документами и разграничение прав доступа. Инфоклиника.RU и Medesk ближе всего к комплексным медицинским информационным системам, ONDOC делает акцент на пациентском личном кабинете, а Cliniccards демонстрирует специализированный подход к ведению карточек пациентов и расписания в стоматологической клинике.

По результатам сравнения можно выделить функции, которые целесообразно учитывать при проектировании веб-приложения для частной клиники. Система должна поддерживать самостоятельную запись пациента на приём, хранение истории посещений, работу врача с медицинской картой, администрирование расписания и защиту данных с помощью ролевой модели. При этом рассмотренные аналоги являются закрытыми коммерческими продуктами, поэтому для курсового проекта важно не копировать готовое решение, а самостоятельно спроектировать структуру данных, пользовательские сценарии и архитектуру приложения.

По итогам анализа к разрабатываемому приложению предъявляются следующие требования:

1. Наличие нескольких ролей пользователей: пациент, врач и администратор.
2. Регистрация и авторизация пользователей.
3. Управление расписанием врачей и доступными временными интервалами.
4. Создание, изменение и отмена записи на приём.
5. Ведение медицинской карты пациента и истории посещений.

6. Загрузка, просмотр и выгрузка файлов, связанных с медицинскими документами.
7. Личный кабинет пациента с просмотром записей и медицинской информации.
8. Административный интерфейс для управления пользователями, врачами, услугами и записями.

1.2 Анализ программных инструментов разработки веб-приложений

При выборе программных инструментов необходимо учитывать специфику предметной области. Приложение для частной клиники работает с персональными и медицинскими данными, поэтому важны надёжная аутентификация, разграничение прав доступа, корректная работа с базой данных, защита форм, удобная разработка CRUD-интерфейсов и возможность дальнейшего развёртывания на сервере.

Для выбора технологической основы рассмотрены несколько распространённых стеков веб-разработки. Сравнение выполнено не по отдельным языкам программирования, а по целостным наборам инструментов, так как серверная часть, шаблонизация, база данных и средства интерфейса должны совместно обеспечивать реализацию требований проекта.

Таблица 1.2 — Анализ существующих стеков разработки веб-приложений

Стек	Состав и назначение	Преимущества для проекта клиники	Ограничения и риски	Итоговая оценка
Python + Flask + PostgreSQL + Bootstrap	Flask отвечает за маршрутизацию, обработку запросов, шаблоны Jinja2 и подключение расширений. PostgreSQL хранит пользователей, врачей, расписание, записи, медицинские карты и документы. Bootstrap используется для форм, таблиц и адаптивной вёрстки [5–8].	Стек даёт понятную архитектуру и позволяет явно реализовать роли, маршруты, модели данных, CRUD-интерфейсы и загрузку файлов. Flask не навязывает лишнюю структуру, поэтому хорошо подходит для учебного проектирования и поэтапной реализации. PostgreSQL удобен для связанных медицинских данных.	Часть механизмов нужно подключать и настраивать самостоятельно: авторизацию, миграции, защиту форм, проверку прав доступа и обработку файлов. Требуется дисциплина в организации структуры проекта.	Оптимальный вариант для проекта на Python: простой, гибкий и достаточный по функциональности.

Продолжение таблицы 1.2

Стек	Состав и назначение	Преимущества для проекта клиники	Ограничения и риски	Итоговая оценка
Python + Django + PostgreSQL	Django включает MVC/MVT-структуру, ORM, административную панель, систему пользователей, формы, миграции и встроенные механизмы безопасности [9].	Быстро создаются модели, админ-панель и типовые CRUD-операции. Стек хорошо подходит для крупных информационных систем с большим количеством сущностей и строгой структурой.	Для курсового проекта может быть избыточным: значительная часть логики скрыта во встроенных механизмах. Сложнее показать самостоятельное проектирование маршрутов и пользовательских сценариев.	Надёжный промышленный вариант, но менее наглядный для демонстрации собственной архитектуры.
Node.js + Express + React/Vue + PostgreSQL или MongoDB	Express реализует серверное API, React или Vue формируют клиентское SPA-приложение, база данных хранит доменные сущности. Обычно стек предполагает разделение frontend и backend [10–12].	Подходит для интерактивного интерфейса, динамического личного кабинета, API и последующего масштабирования клиентской части. Удобен, когда требуется сложный frontend.	Увеличивает объём работы: нужно разрабатывать API, клиентское состояние, сборку frontend, авторизацию токенами и обмен данными между частями приложения. Для базового прототипа это усложняет реализацию.	Подходит для SPA, но избыточен для отчётного прототипа с серверными шаблонами.
PHP + Laravel + MySQL/PostgreSQL	Laravel предоставляет MVC-структуру, маршруты, шаблоны Blade, ORM Eloquent, миграции, middleware, валидацию форм и средства аутентификации [13].	Удобен для классических веб-приложений с личными кабинетами, формами, административной частью и CRUD-интерфейсами. Имеет развитую экосистему готовых инструментов.	Требуется использование PHP и знания экосистемы Laravel. Если проект ориентирован на Python, переход на Laravel не даёт принципиального преимущества и усложняет сопровождение.	Практичная альтернатива, но не соответствует выбранному языку реализации.
Java + Spring Boot + PostgreSQL	Spring Boot используется для серверной части, REST-контроллеров, сервисного слоя, безопасности, работы с БД через JPA/Hibernate и построения корпоративных приложений [14].	Подходит для сложных систем, где важны строгая типизация, многоуровневая архитектура, масштабируемость, тестируемость и развитые механизмы безопасности.	Имеет высокий порог входа, требует больше конфигурации и более сложной структуры проекта. Для учебного прототипа частной клиники может быть слишком тяжёлым.	Надёжный, но избыточный стек для заданного объёма работы.
ASP.NET Core + PostgreSQL/SQL Server	ASP.NET Core позволяет создавать MVC-приложения и API, подключать Identity, Entity Framework Core и работать с реляционными базами данных [15].	Хорошо подходит для корпоративных веб-систем, поддерживает строгую архитектуру, производительность и развитые инструменты безопасности.	Требуется использование C# и отдельной экосистемы разработки. Для проекта, ориентированного на Python, усложняет реализацию и не совпадает с планируемым стеком.	Сильный промышленный вариант, но нецелесообразен для выбранной реализации.

По результатам анализа выбран стек Python + Flask + PostgreSQL + Bootstrap. Он соответствует задачам курсового проекта и позволяет реализовать приложение без избыточного усложнения архитектуры. Flask обеспечивает маршрутизацию, шаблоны и возможность подключать только необходимые

расширения: авторизацию пользователей, ORM для работы с базой данных, обработку форм, миграции и загрузку файлов. PostgreSQL подходит для реляционной модели предметной области, где пользователь, пациент, врач, расписание, запись на приём, посещение и медицинский документ связаны между собой. Bootstrap и базовые технологии HTML, CSS и JavaScript позволяют подготовить адаптивный интерфейс личного кабинета, расписания, медицинской карты и административных страниц.

Выбор Flask обоснован тем, что он даёт возможность явно показать структуру веб-приложения: маршруты, модели, формы, шаблоны, роли пользователей и логику доступа. В отличие от Django, он не скрывает значительную часть архитектуры за готовыми механизмами; в отличие от SPA-стеков на Node.js, не требует отдельной разработки сложного frontend-приложения; по сравнению с Laravel, Spring Boot и ASP.NET Core лучше соответствует выбранному языку и учебному объёму проекта. Следовательно, стек Python + Flask является наиболее рациональным вариантом для проектирования и последующей реализации веб-приложения частной клиники.

В качестве инструмента для обеспечения кроссплатформенности и упрощения развертывания веб-приложения обоснован выбор технологии контейнеризации Docker. Использование Docker позволяет упаковать приложение клиники, его зависимости (Flask-расширения, СУБД) и начальную конфигурацию в единый изолированный контейнер, который гарантирует идентичную работу системы на любом компьютере или хост-сервере независимо от установленной операционной системы и локальных библиотек. Это минимизирует риски несовместимости версий интерпретатора Python, исключает конфликты пакетов, ускоряет развертывание и существенно упрощает масштабирование информационной системы медицинского учреждения в процессе промышленной эксплуатации.

1.3 Формулировка цели и задач работы

Целью курсового проекта является разработка веб-приложения для частной клиники, обеспечивающего онлайн-запись пациентов на приём, ведение медицинских карт, работу с расписанием врачей и предоставление пользователям личных кабинетов с разграничением прав доступа.

Для достижения поставленной цели в рамках пояснительной записки

необходимо решить следующие задачи:

1. Провести анализ предметной области и рассмотреть существующие программные продукты для автоматизации клиник.
2. Определить функциональные требования к веб-приложению на основе анализа аналогов и задания на курсовой проект.
3. Выполнить полный анализ существующих стеков веб-разработки и обосновать выбор Python + Flask.
4. Проанализировать целевую аудиторию и роли пользователей.
5. Описать функциональность приложения через диаграмму вариантов использования и пользовательские истории.
6. Разработать ER-диаграмму, логическую и физическую схемы базы данных.
7. Подготовить макеты страниц личных кабинетов и основных пользовательских сценариев.

Итогом выполнения курсового проекта должно стать описание и проектная основа веб-приложения, демонстрирующего основные процессы цифрового взаимодействия частной клиники с пациентами. Подготовленные результаты позволяют перейти к программной реализации системы на стеке Python + Flask, закрепить навыки проектирования веб-систем, работы с базами данных, моделирования пользовательских ролей и оформления пояснительной записки в соответствии с требованиями ГОСТ 7.32-2017 [16].

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ

2.1 Анализ целевой аудитории

Разрабатываемое веб-приложение ориентировано на пользователей, которые участвуют в процессе оказания медицинских услуг в частной клинике. Для корректного проектирования интерфейса и разграничения доступа выделены три основные роли: пациент, врач и администратор. Дополнительно система должна учитывать неавторизованного посетителя, который может ознакомиться с общими сведениями о клинике, но не имеет доступа к медицинским данным.

Таблица 2.1 — Характеристика целевой аудитории веб-приложения

Роль	Основные потребности	Типовые действия в системе
Неавторизованный посетитель	Получение общей информации о клинике, услугах и врачах.	Просмотр публичных страниц, переход к регистрации или форме входа.
Пациент	Быстрая запись на приём, просмотр расписания, доступ к истории посещений и медицинским документам.	Регистрация, вход в личный кабинет, выбор врача и услуги, создание или отмена записи, просмотр рекомендаций врача.
Врач	Удобный просмотр расписания, доступ к медицинской карте пациента, фиксация результатов приёма.	Просмотр списка приёмов, открытие карты пациента, добавление жалоб, диагноза, назначений и файлов.
Администратор	Управление расписанием клиники, контроль записей, поддержка справочников и пользователей.	Создание врачей и услуг, настройка графика работы, подтверждение или перенос записей, управление учётными записями.

Для пациента интерфейс должен быть простым и понятным: основные действия должны выполняться за минимальное количество шагов. Для врача важны быстрый доступ к расписанию и медицинской карте пациента. Для администратора требуется более полный набор инструментов, так как эта роль отвечает за поддержание актуальности данных в системе.

2.2 Описание функциональности приложения

Функциональность приложения определяется через действия пользователей и системные ограничения. Общая диаграмма вариантов использования представлена на рисунке 2.1. Диаграмма отражает связь между тремя основными ролями и ключевыми действиями: авторизацией, записью на приём, управлением расписанием, ведением медицинских карт и просмотром истории посещений.

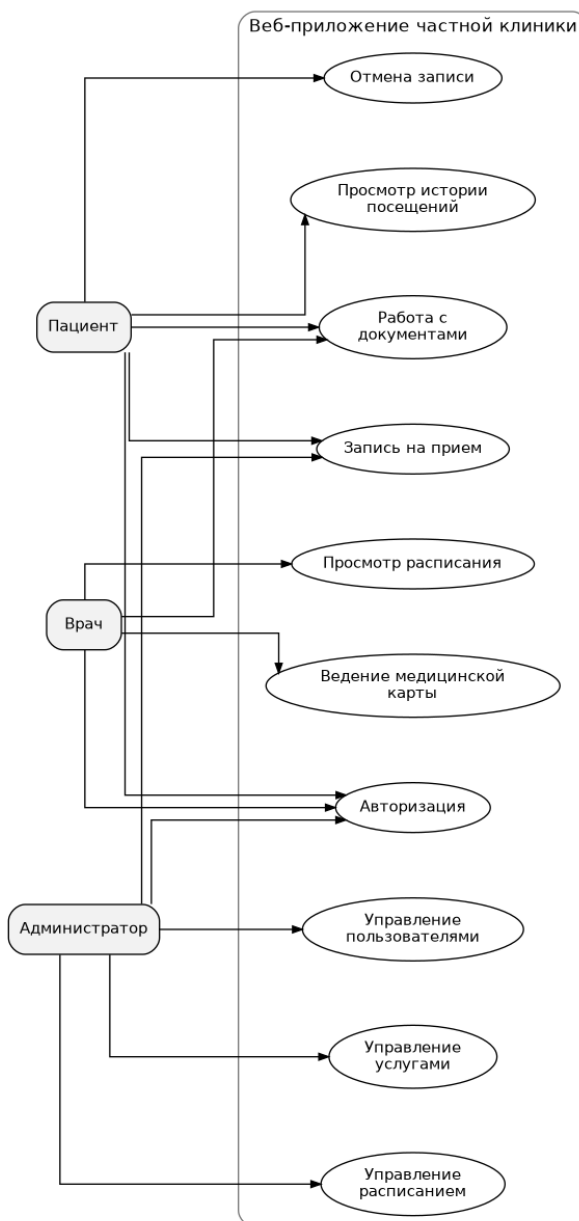


Рисунок 2.1 — Диаграмма вариантов использования веб-приложения

Ключевые пользовательские сценарии можно описать с помощью User Story:

1. Я, как пациент, хочу выбрать врача, услугу и свободное время, чтобы

самостоятельно записаться на приём без звонка в клинику.

2. Я, как пациент, хочу видеть историю приёмов и рекомендации врача, чтобы иметь доступ к медицинской информации после посещения.
3. Я, как врач, хочу просматривать своё расписание на день, чтобы заранее видеть список пациентов и время приёма.
4. Я, как врач, хочу заполнять медицинскую карту пациента, чтобы фиксировать результаты приёма и назначения.
5. Я, как администратор, хочу управлять расписанием врачей, услугами и записями, чтобы обеспечивать корректную работу клиники.
6. Я, как администратор, хочу иметь возможность выгружать или прикреплять документы, чтобы поддерживать актуальность медицинской информации пациента.

На основании анализа сценариев сформирован перечень функциональных требований, представленный в таблице 2.2.

Таблица 2.2 — Функциональные требования к веб-приложению

Номер	Требование	Роль
1	Система должна обеспечивать регистрацию, вход и выход пользователей.	Пациент, врач, администратор
2	Система должна разграничивать доступ к страницам и операциям по ролям.	Пациент, врач, администратор
3	Пациент должен иметь возможность просматривать список врачей, услуг и свободных временных интервалов.	Пациент
4	Пациент должен иметь возможность создать, просмотреть или отменить запись на приём.	Пациент
5	Врач должен иметь возможность просматривать собственное расписание и карточку приёма.	Врач
6	Врач должен иметь возможность заполнять медицинскую карту пациента по итогам приёма.	Врач

Продолжение таблицы 2.2

Номер	Требование	Роль
7	Администратор должен управлять врачами, услугами, расписанием, пользователями и записями.	Администратор
8	Система должна поддерживать загрузку, просмотр и выгрузку медицинских документов.	Врач, пациент, администратор
9	Система должна отображать пациенту личный кабинет с будущими записями, историей посещений и медицинской картой.	Пациент

Кроме функциональных требований, для приложения важны нефункциональные требования: защита персональных данных, устойчивость к ошибочным действиям пользователя, адаптивность интерфейса, понятная навигация и возможность резервного копирования базы данных.

2.3 Проектирование модели данных

Модель данных должна отражать связи между основными объектами предметной области. Ключевыми сущностями являются пользователь, врач, пациент, услуга, расписание, запись на приём, медицинская карта, посещение и медицинский документ. Упрощённая ER-диаграмма представлена на рисунке 2.2.

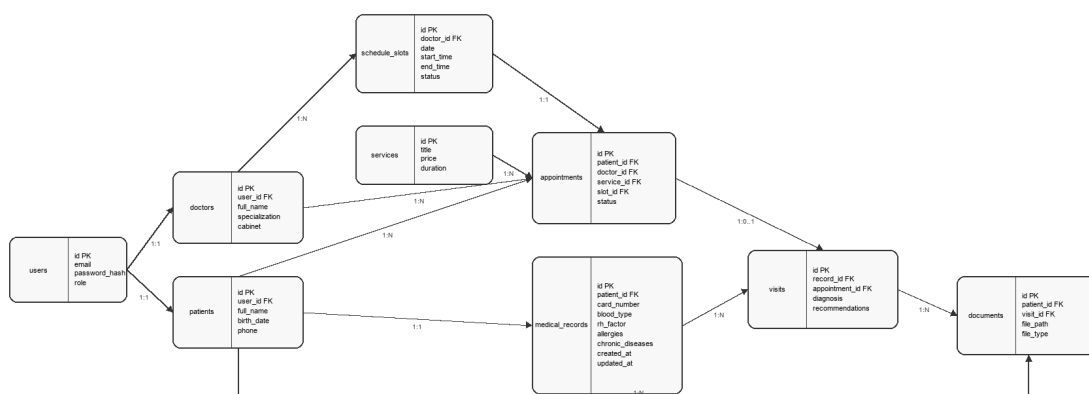


Рисунок 2.2 — Упрощённая ER-диаграмма базы данных

Логическая схема базы данных строится на следующих правилах: один пользователь имеет одну роль; пациент связан с одной учётной записью; врач связан с одной учётной записью и может иметь множество слотов расписания; запись на приём связывает пациента, врача, услугу и свободный временной

интервал; медицинская карта принадлежит пациенту и содержит персональные медицинские показатели (номер карты, группа крови, резус-фактор, аллергические реакции и противопоказания, хронические заболевания), а также связана с историей посещений; документы могут быть связаны с пациентом и конкретным посещением.

Таблица 2.3 — Логическая схема данных

Связь	Кратность	Описание
users - patients	1:1	Учётная запись пациента содержит данные для входа, а профиль пациента хранит персональную информацию.
users - doctors	1:1	Учётная запись врача используется для входа и разграничения доступа к расписанию и медицинским картам.
doctors - schedule_slots	1:N	Один врач имеет множество временных интервалов расписания.
patients - appointments	1:N	Один пациент может иметь несколько записей на приём.
appointments - visits	1:0..1	После завершения приёма запись может быть связана с результатом посещения.
patients - medical_records	1:1	Медицинская карта создаётся для конкретного пациента.
visits - documents	1:N	К одному посещению могут быть прикреплены результаты анализов, заключения или иные файлы.

Основные таблицы базы данных представлены в таблице 2.4. В рамках курсового проекта такая структура достаточна для реализации базовой логики приложения и может быть расширена при дальнейшем развитии системы.

Таблица 2.4 — Основные таблицы базы данных

Таблица	Ключевые поля	Назначение
users	id, email, password_hash, role, created_at	Хранение учётных записей и роли пользователя.

Продолжение таблицы 2.4

Таблица	Ключевые поля	Назначение
patients	id, user_id, full_name, birth_date, phone	Данные пациента, связанные с учётной записью.
doctors	id, user_id, full_name, specialization, cabinet	Данные врача, специальность и кабинет приёма.
services	id, title, description, price, duration_minutes	Справочник медицинских услуг.
schedule_slots	id, doctor_id, date, start_time, end_time, status	Доступные временные интервалы врача.
appointments	id, patient_id, doctor_id, service_id, slot_id, status	Записи пациентов на приём и их статусы.
medical_records	id, patient_id, card_number, blood_type, rh_factor, allergies, chronic_diseases, created_at, updated_at	Электронная медицинская карта пациента с указанием номера, группы крови, резус-фактора, аллергических реакций и хронических заболеваний.
visits	id, record_id, appointment_id, complaints, diagnosis, recommendations	Результаты конкретного посещения врача.
documents	id, patient_id, visit_id, file_path, file_type	Прикреплённые медицинские файлы и результаты исследований.

Физическая структура одной из ключевых таблиц - таблицы записей на приём - представлена в таблице 2.5. Она показывает, какие типы данных и ограничения должны быть использованы при реализации базы данных в PostgreSQL.

Таблица 2.5 — Структура таблицы записей на приём (appointments)

Поле	Тип	Ограничение	Описание
id	SERIAL	PRIMARY KEY	Уникальный идентификатор записи.
patient_id	INTEGER	FOREIGN KEY	Ссылка на пациента.
doctor_id	INTEGER	FOREIGN KEY	Ссылка на врача.
service_id	INTEGER	FOREIGN KEY	Ссылка на выбранную услугу.

Продолжение таблицы 2.5

Поле	Тип	Ограничение	Описание
slot_id	INTEGER	FOREIGN KEY, UNIQUE	Ссылка на временной интервал расписания; ограничение UNIQUE предотвращает двойную запись на один слот.
status	VARCHAR(20)	NOT NULL	Статус записи: создана, подтверждена, отменена, завершена.
created_at	TIMESTAMP	NOT NULL	Дата и время создания записи.

Физическая структура электронной медицинской карты пациента представлена в таблице 2.6. В ней отражены все ключевые медицинские параметры пациента, необходимые врачу для принятия решений и назначения безопасного лечения.

Таблица 2.6 — Структура таблицы медицинских карт (medical_records)

Поле	Тип	Ограничение	Описание
id	SERIAL	PRIMARY KEY	Уникальный идентификатор медицинской карты.
patient_id	INTEGER	FOREIGN KEY, UNIQUE	Ссылка на пациента; ограничение UNIQUE гарантирует связь 1:1.
card_number	VARCHAR(50)	NOT NULL, UNIQUE	Уникальный номер медицинской карты.
blood_type	VARCHAR(10)	—	Группа крови пациента (например, 0(I), A(II)).
rh_factor	VARCHAR(5)	—	Резус-фактор (Rh+, Rh-).
allergies	TEXT	—	Перечень аллергических реакций и лекарственной непереносимости.

Продолжение таблицы 2.6

Поле	Тип	Ограничение	Описание
chronic_diseases	TEXT	—	Зарегистрированные хронические заболевания пациента.
created_at	TIMESTAMP	NOT NULL	Дата и время создания карты.
updated_at	TIMESTAMP	NOT NULL	Дата и время последнего обновления данных.

Также необходимо предусмотреть хранение файлов. В базе данных должен сохраняться не сам файл, а путь к нему, тип файла, владелец документа и связь с посещением. Такой подход упрощает работу с базой данных и позволяет организовать отдельное хранилище загруженных медицинских документов.

2.4 Разработка макетов страниц

На этапе проектирования интерфейса определены макеты основных страниц веб-приложения, которые обеспечивают выполнение всех пользовательских сценариев. Для наглядности и соблюдения единой дизайн-системы макеты страниц спроектированы в виде схематичных блоков (Wireframe).

Публичная главная страница веб-приложения (рисунок 2.3) содержит общую информацию о частной клинике, её ключевых медицинских направлениях, контактах и адресе. Верхнее меню предоставляет ссылки для быстрой навигации и переход к авторизации. Центральная часть отведена под промо-баннер с интерактивной кнопкой «Записаться на приём», которая перенаправляет пользователя на страницу онлайн-записи.

Страницы авторизации и регистрации пользователей (рисунок 2.4) спроектированы в виде двух отдельных форм на едином экране. Слева располагается лаконичная форма входа (авторизации), содержащая поля ввода адреса электронной почты (Email) и пароля, а также кнопку отправки. Справа представлена развернутая форма регистрации нового пациента, запрашивающая ФИО, дату рождения, номер телефона, Email и будущий пароль. В нижней части (в подвале) размещено предупреждение о согласии на обработку

Шапка: Логотип | О клинике | Услуги | Специалисты | Личный кабинет [Войти]

Промо-баннер: Современная клиника для всей семьи
[Кнопка: Записаться на приём]

Направление: Терапия
Приемы терапевта,
первичная диагностика

Направление: Кардиология
Консультации, ЭКГ,
УЗИ сердца

Направление: Анализы
Забор крови, ПЦР,
экспресс-тесты

Адрес: г. Москва, ул. Автозаводская, 16 | Телефон: +7 (495) 123-45-67

Подвал: © 2026 Частная клиника. Все права защищены.

Рисунок 2.3 — Макет структуры публичной главной страницы веб-приложения

персональных данных пациентов в соответствии с Федеральным законом № 152-ФЗ.

Шапка: Логотип | [На главную страницу клиники]

Страница авторизации и регистрации

Форма входа (Авторизация)

[Поле ввода: Email]
[Поле ввода: Пароль]

[Кнопка: Войти в кабинет]

Форма регистрации пациента

[Поле ввода: ФИО пациента]
[Поле ввода: Дата рождения]
[Поле ввода: Номер телефона]
[Поле ввода: Email]
[Поле ввода: Пароль]

[Кнопка: Зарегистрироваться]

Подвал: Защита персональных данных пациентов согласно ФЗ-152 | © 2026

Рисунок 2.4 — Макет структуры страниц регистрации и входа в систему

Страница онлайн-записи на приём (рисунок 2.5) представляет собой интерактивную форму, разделенную на несколько последовательных шагов. Пациент выбирает медицинское направление (специализацию) и конкретного врача, указывает требуемую услугу, а затем на интерактивном календаре отме-

чает свободную дату и время приёма. В нижней части страницы отображается резюме записи с кнопкой окончательного подтверждения, что минимизирует вероятность ошибок при вводе.

Шапка: Логотип | Пациент: Амплеенков Д.О. | Личный кабинет | [Выйти]

Форма онлайн-записи на приём

Шаг 1: Направление и врач
[Выбрать специальность]
[Выбрать врача]

Шаг 2: Выберите услугу
[Выбрать услугу]
Длительность: 30 мин., Цена: 1500 руб.

Шаг 3: Выберите дату и свободное время
[Интерактивный календарь]
[Слоты: 09:00, 09:30, 10:00 (занят), 10:30, 11:00]

Подтверждение записи
Выбран врач: Иванов И.И., услуга: Прием терапевта
Дата и время: 28.05.2026 в 09:30
[Кнопка: Подтвердить запись]

Подвал: Служба поддержки клиники | © 2026

Рисунок 2.5 — Макет структуры страницы онлайн-записи на приём

Личный кабинет пациента (рисунок 2.6) выступает персональным центром управления медицинскими услугами. Здесь отображаются сведения об активных и прошедших приёмах, история визитов к врачам, рекомендации по лечению, а также прикрепленные медицинские файлы и результаты исследований.

Личный кабинет врача (рисунок 2.7) разделен на две основные функциональные зоны. Слева находится сетка расписания на текущий день с указанием ФИО пациентов и статусов приёмов (принят, в процессе, ожидает). Справа расположена форма активного приёма, позволяющая врачу оперативно вносить жалобы пациента, анамнез, формулировать диагноз, выписывать назначения и рецепты, а также прикреплять сканы медицинских документов и результаты исследований.

Административная панель веб-приложения (рисунок 2.8) спроектирована по классическому двухколонному шаблону. Левая колонка представляет собой боковое меню для переключения между разделами управления (врачи,

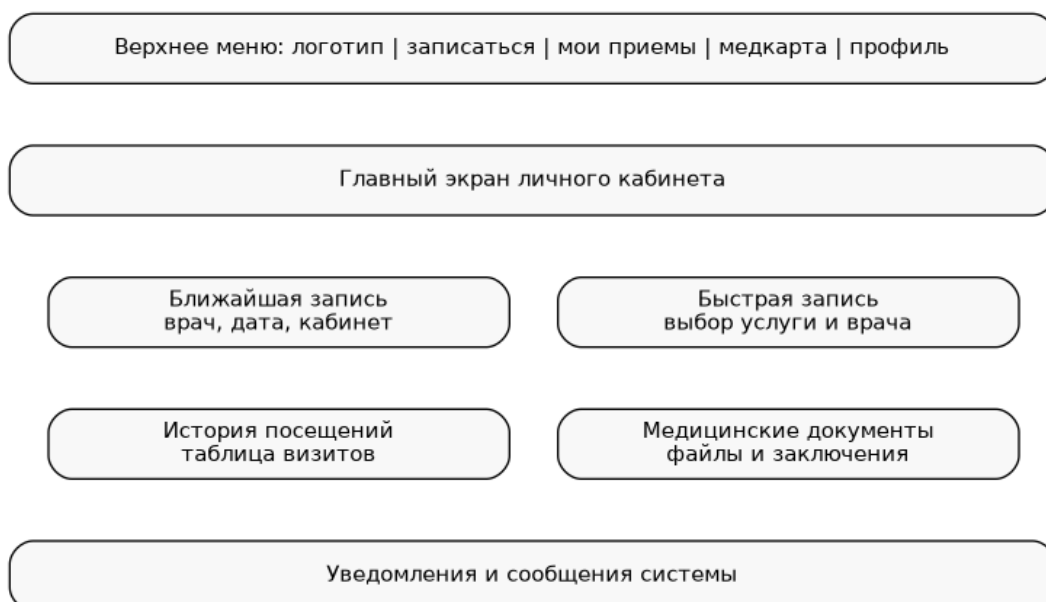


Рисунок 2.6 — Макет структуры личного кабинета пациента



Рисунок 2.7 — Макет структуры личного кабинета врача

услуги, расписание, пациенты, записи). Правая (основная) часть отображает текущую рабочую область. На примере раздела управления расписанием показаны возможности фильтрации данных по дате и врачам, табличный вид слотов расписания с быстрыми действиями (редактирование, удаление, отмена записи) и кнопка создания новых слотов.

При проектировании интерфейса используются следующие принципы:

1. Основная навигация должна быть доступна с любой страницы после входа в систему.

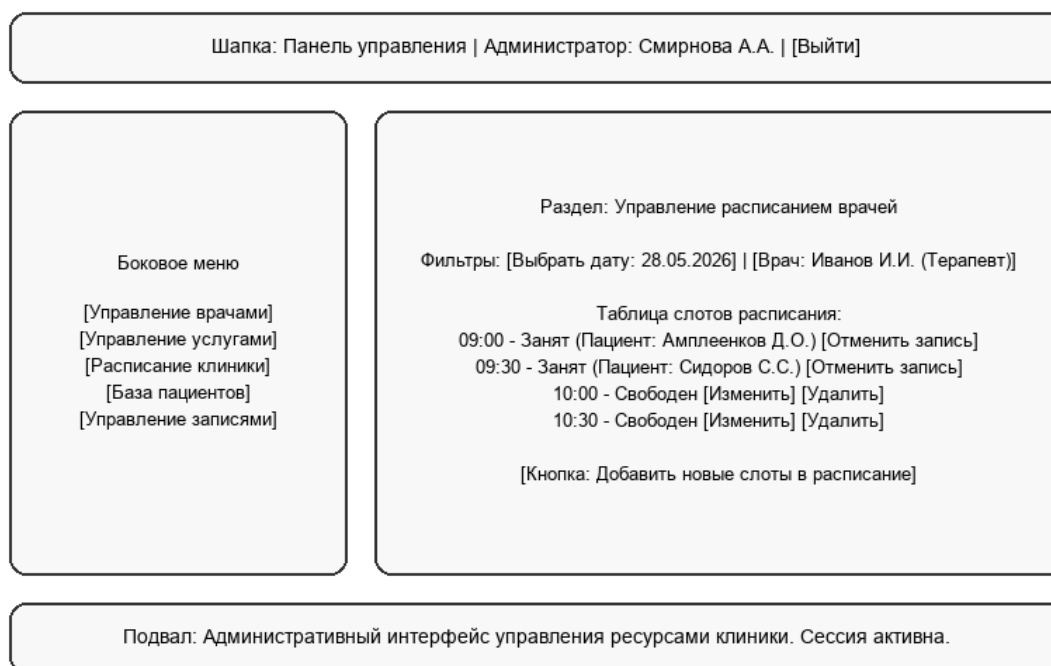


Рисунок 2.8 — Макет структуры панели администратора

2. Формы должны содержать минимально необходимый набор полей.
3. Таблицы расписания и записей должны поддерживать фильтрацию по дате, врачу и статусу.
4. Действия, связанные с изменением медицинской карты, должны быть доступны только врачу и администратору.
5. Пациент не должен видеть данные других пациентов.
6. После успешного действия пользователь должен получать понятное уведомление.

2.5 Контейнеризация и развертывание с использованием Docker

Для обеспечения кроссплатформенности, воспроизводимости окружения и упрощения процесса развертывания веб-приложения на различных серверах была спроектирована система контейнеризации с использованием технологии Docker. Использование контейнеров позволяет полностью изолировать программную среду приложения частной клиники, гарантируя одинаковую работу веб-ресурса на стадиях разработки, локального тестирования и промышленной эксплуатации.

Спроектированная архитектура контейнеризации базируется на следующих файлах конфигурации:

1. `Dockerfile` — содержит набор последовательных инструкций для сборки Docker-образа приложения.
2. `.dockerignore` — определяет список файлов и директорий, которые должны быть исключены из контекста сборки Docker-контейнера.

В соответствии с требованиями безопасности и оптимизации дискового пространства, в сборку Docker-образа включается только программный код приложения (папка `clinic_app`). Все файлы, относящиеся к исходным кодам отчёта (папка `report`, исходные `.tex` файлы, скомпилированные `.pdf` документы, сборочный `Makefile` и временные LaTeX-файлы компиляции), а также локальные виртуальные окружения Python (`.venv`) и кэши байт-кода (`__pycache__`) явно исключаются через конфигурационный файл `.dockerignore`. Это позволяет уменьшить размер результирующего Docker-образа в несколько раз и предотвращает попадание нерелевантной документации во внутреннюю среду контейнера.

В качестве базового образа выбран официальный оптимизированный образ `python:3.11-slim`, отличающийся минимальным размером и высокой производительностью. Процесс сборки и развертывания состоит из следующих шагов:

1. Установка рабочей директории в контейнере (`/app`).
2. Копирование списка зависимостей `clinic_app/requirements.txt` и запуск пакетного менеджера `pip` для установки необходимых Python-библиотек.
3. Копирование исходного кода приложения `clinic_app` в контейнер.
4. Запуск скрипта автоматического первоначального наполнения базы данных (`python seed.py`) для создания локальной БД SQLite и её заполнения тестовыми записями (врачи, услуги, расписание).
5. Открытие сетевого порта 5000 и запуск приложения с помощью команды `python app.py`.

Для локальной сборки и запуска веб-приложения в контейнере используются стандартные команды Docker CLI:

Сборка Docker-образа

```
docker build -t clinic-app .
```

Запуск контейнера с пробросом портов

```
docker run -p 5000:5000 clinic-app
```

Использование Docker позволяет развернуть готовую к работе информационную систему клиники с полностью инициализированной базой данных за несколько секунд на любом хосте без необходимости предустановки интерпретатора Python и СУБД.

Для автоматизации процессов оркестрации, управления сетевыми ресурсами и организации постоянного хранения медицинских данных (персистентности) дополнительно разработан конфигурационный файл `docker-compose.yml`. Он описывает параметры запуска веб-контейнера и настраивает проброс сетевых портов, а также создает постоянные тома (Volumes) для монтирования папки локальной базы данных (`clinic_app/instance`) и директории загрузки результатов осмотра и медицинских файлов пациентов (`clinic_app/static/uploads`) на хост-компьютере. Это предотвращает потерю накопленной медицинской информации при перезагрузке, остановке или удалении контейнера.

Запуск всего приложения с инициализированной БД и персистентными томами выполняется с помощью одной консольной команды:

```
docker-compose up --build
```

Таким образом, проектирование веб-приложения включает выделение целевой аудитории, описание функциональности через диаграмму вариантов использования и пользовательские истории, разработку модели данных, логической и физической схемы БД, а также подготовку макетов страниц. Эти результаты являются основой для последующей разработки программной реализации.

ГЛАВА 3. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ ВЕБ-ПРИЛОЖЕНИЯ

3.1 Структура программного решения

Программная реализация веб-приложения для частной медицинской клиники выполнена на объектно-ориентированном языке программирования Python с использованием микрофреймворка Flask. Выбор Flask обусловлен его легковесностью, модульностью и простотой интеграции со сторонними библиотеками для работы с базами данных (Flask-SQLAlchemy) и организации сессий пользователей (Flask-Login).

Проект имеет следующую структуру директорий и файлов:

1. `app.py` - основной файл приложения, содержащий инициализацию Flask-сервера, настройку обработчиков маршрутов (endpoints), логику обработки запросов и авторизации.
2. `models.py` - модуль описания структуры базы данных на уровне объектно-реляционного отображения (ORM SQLAlchemy).
3. `config.py` - конфигурационный класс приложения, определяющий настройки безопасности (секретный ключ сессии), путь к базе данных SQLite/PostgreSQL, параметры загрузки документов и лимиты на размер файлов.
4. `seed.py` - скрипт для первоначального создания таблиц базы данных и наполнения их демонстрационными данными (учетными записями тестовых пользователей различных ролей, списком услуг и расписанием врачей).
5. `requirements.txt` - перечень внешних зависимостей проекта и версий библиотек для сборки окружения.
6. `templates/` - каталог HTML-шаблонов страниц с разметкой на базе шаблонизатора Jinja2.
7. `static/` - папка для статических ресурсов приложения, включающая стили CSS, скрипты JavaScript, иконки Bootstrap Icons и подкаталог `uploads/` для загрузки медицинских заключений и файлов пациентов.

Конфигурационный файл `config.py` обеспечивает адаптивность приложения к среде развертывания: при наличии в переменных окружения строки подключения `DATABASE_URL` приложение подключается к внешней СУБД PostgreSQL (промышленный режим), иначе используется локальный файл базы данных `clinic.db` через встроенную поддержку SQLite (режим разработки).

3.2 Модели данных и ORM-маппинг

Для управления персистентным состоянием и выполнения запросов к СУБД в приложении применена технология ORM (Object-Relational Mapping) на базе Flask-SQLAlchemy. Описание логических сущностей и связей выполнено в виде Python-классов, наследующих базовый класс декларативного описания `db.Model`.

Основные классы моделей, реализованные в модуле `models.py`:

1. `User` - учётная запись пользователя. Содержит поля `id`, `email`, `password_hash` (хэш пароля на базе алгоритма `scrypt`) и `role` (роль: `'patient'`, `'doctor'` или `'admin'`). Реализует методы `set_password` для хэширования и `check_password` для проверки введённого пароля.
2. `Patient` - профиль пациента. Связан отношением 1:1 с учётной записью `User` с использованием каскадного удаления `cascade="all, delete-orphan"`. Содержит персональные данные (ФИО, дата рождения, телефон).
3. `Doctor` - профиль врача. Связан отношением 1:1 с учётной записью `User`. Дополнительно содержит поля специализации (ссылка на `Specialization`) и номера кабинета приёма `cabinet`.
4. `Specialization` - справочник врачебных специальностей (например, терапевт, кардиолог, невролог).
5. `Service` - перечень предоставляемых услуг с указанием названия, описания, цены и длительности в минутах.
6. `ScheduleSlot` - интервалы расписания приёма. Содержит ссылку на `Doctor`, дату, время начала и окончания интервала, а также статус занятости слота (`'free'`, `'reserved'`).

7. Appointment - запись на приём. Связывает пациента (Patient), врача (Doctor), услугу (Service) и временной интервал расписания (ScheduleSlot). Поле slot_id снабжено ограничением уникальности unique=True, предотвращающим повторное бронирование одного слота.
8. MedicalRecord - электронная медицинская карта. Хранит группу крови, резус-фактор, перечень аллергенов и хронических болезней пациента. Связана 1:1 с пациентом.
9. Visit - протокол осмотра (посещения). Содержит жалобы, диагноз и рекомендации врача. Создаётся для конкретной записи Appointment после выполнения приёма.
10. Document - прикрепленные файлы исследований (картинки, документы PDF). Связан с пациентом и (опционально) с конкретным посещением.

Связи между таблицами настроены с использованием внешних ключей (db.ForeignKey) с поддержкой ограничений целостности БД (ondelete='CASCADE' и ondelete='SET NULL'), что обеспечивает корректную работу транзакций СУБД.

Пример реализации структуры моделей данных на языке Python с использованием библиотеки Flask-SQLAlchemy представлен в листинге 1.

Листинг 1 – Реализация моделей данных в clinic_app/models.py

```
1 from flask_sqlalchemy import SQLAlchemy
2 from flask_login import UserMixin
3 from werkzeug.security import generate_password_hash,
   check_password_hash
4 from datetime import datetime
5
6 db = SQLAlchemy()
7
8 class User(db.Model, UserMixin):
9     __tablename__ = 'users'
10    id = db.Column(db.Integer, primary_key=True)
11    email = db.Column(db.String(120), unique=True, nullable=False)
12    password_hash = db.Column(db.String(256), nullable=False)
13    role = db.Column(db.String(20), nullable=False) # 'patient', 'doctor', 'admin'
```

```

14     created_at = db.Column(db.DateTime, default=datetime.
        utcnow)
15
16     patient = db.relationship('Patient', backref='user',
        uselist=False, cascade="all, delete-orphan")
17     doctor = db.relationship('Doctor', backref='user',
        uselist=False, cascade="all, delete-orphan")
18
19     def set_password(self, password):
20         self.password_hash = generate_password_hash(password
        )
21
22     def check_password(self, password):
23         return check_password_hash(self.password_hash,
        password)
24
25 class Patient(db.Model):
26     __tablename__ = 'patients'
27     id = db.Column(db.Integer, primary_key=True)
28     user_id = db.Column(db.Integer, db.ForeignKey('users.id'
        , ondelete='CASCADE'), unique=True, nullable=False)
29     full_name = db.Column(db.String(150), nullable=False)
30     birth_date = db.Column(db.Date, nullable=False)
31     phone = db.Column(db.String(20), nullable=False)
32
33     appointments = db.relationship('Appointment', backref='
        patient', lazy=True, cascade="all, delete-orphan")
34     medical_record = db.relationship('MedicalRecord',
        backref='patient', uselist=False, cascade="all, delete-
        orphan")
35
36 class Appointment(db.Model):
37     __tablename__ = 'appointments'
38     id = db.Column(db.Integer, primary_key=True)
39     patient_id = db.Column(db.Integer, db.ForeignKey('
        patients.id', ondelete='CASCADE'), nullable=False)
40     doctor_id = db.Column(db.Integer, db.ForeignKey('doctors
        .id'), nullable=False)
41     service_id = db.Column(db.Integer, db.ForeignKey('
        services.id'), nullable=False)
42     slot_id = db.Column(db.Integer, db.ForeignKey('
        schedule_slots.id'), unique=True, nullable=False)
43     status = db.Column(db.String(20), nullable=False,
        default='created')
44     created_at = db.Column(db.DateTime, default=datetime.

```

```
utcnow)
45     visit = db.relationship('Visit', backref='appointment',
                             uselist=False, cascade="all, delete-orphan")
```

3.3 Логика маршрутизации и бизнес-логика

Маршрутизация запросов и бизнес-логика реализованы в виде контроллеров Flask в файле `app.py`. Доступ к защищённым страницам регулируется расширением Flask-Login. Ограничение доступа к страницам личных кабинетов реализовано с помощью декоратора `@login_required` и проверок роли текущего авторизованного пользователя `current_user.role`.

Основные функциональные маршруты приложения:

1. Публичные маршруты:

- `/` (функция `index`) - отображение главной страницы с направлениями работы и списком услуг клиники;
- `/auth` (функция `auth_page`) - страница аутентификации и регистрации;
- `/login` и `/register` (POST-обработчики) - выполняют валидацию входных данных, проверку уникальности email, авторизацию сессии с помощью `login_user` и перенаправление пользователя в личный кабинет в зависимости от его роли.

2. Личный кабинет пациента:

- `/cabinet` (функция `patient_cabinet`) - собирает информацию о профиле пациента, его электронной медицинской карте, а также списки будущих записей на приём и архив прошедших визитов с рекомендациями врачей;
- `/booking` (функция `booking_page`) - страница онлайн-записи на приём. Передаёт в шаблон списки направлений, врачей и услуг;
- `/booking/get_slots` - API-маршрут (возвращает JSON), используемый динамическим JavaScript на клиенте для получения свободных слотов расписания конкретного врача на выбранную дату;

- /booking/create (POST-запрос) - создание записи на приём. Бронирует выбранный слот (переводит его статус в 'reserved'), создаёт объект Appointment и перенаправляет в кабинет.

3. Личный кабинет врача:

- /doctor (функция doctor_cabinet) - отображает расписание приёмов врача на выбранную дату (по умолчанию на текущий день), позволяя фильтровать записи;
- /doctor/visit/<int:appt_id> - страница проведения приёма пациента. Врач вносит текстовые жалобы, диагноз и рекомендации. По завершении создаётся запись Visit, статус приёма обновляется на 'completed', а слот расписания закрывается;
- /doctor/upload_document - обработчик загрузки медицинских файлов. Выполняет проверку расширения файла, сохраняет его в папку static/uploads с безопасным именем и связывает запись Document с пациентом.

4. Панель администратора:

- /admin (функция admin_dashboard) - сводная панель со статистикой работы клиники;
- /admin/doctors и /admin/services - управление справочниками медицинского персонала и услуг (добавление, редактирование, удаление записей);
- /admin/appointments - общий реестр записей на приём с возможностью подтверждения (confirm) или отмены (cancel).

При обработке форм реализован механизм всплывающих уведомлений flash(), информирующий пользователя об успешном завершении операции или возникших ошибках валидации данных.

Пример реализации обработчика маршрута для подтверждения записи пациента на приём с контролем занятости выбранного слота расписания и транзакционной логикой СУБД представлен в листинге 2.

Листинг 2 – Реализация маршрута подтверждения записи в clinic_app/app.py

```
1 @app.route('/confirm_booking', methods=['POST'])
2 @login_required
3 def confirm_booking():
4     slot_id = request.form.get('slot_id')
5     doctor_id = request.form.get('doctor_id')
6     service_id = request.form.get('service_id')
7
8     slot = ScheduleSlot.query.get(slot_id)
9     if not slot or slot.status != 'free':
10         flash('Выбранный временной слот уже занят.', 'error')
11         return redirect(url_for('booking_page'))
12
13     slot_datetime = datetime.combine(slot.date, slot.
start_time)
14     if slot_datetime < datetime.now():
15         flash('Выбранное время приема уже прошло.', 'error')
16         return redirect(url_for('booking_page'))
17
18     try:
19         appt = Appointment(
20             patient_id=current_user.patient.id,
21             doctor_id=int(doctor_id),
22             service_id=int(service_id),
23             slot_id=int(slot_id),
24             status='confirmed'
25         )
26         db.session.add(appt)
27         slot.status = 'booked'
28         db.session.commit()
29
30         flash('Вы успешно записались на приём!', 'success')
31         return redirect(url_for('patient_cabinet'))
32
33     except Exception as e:
34         db.session.rollback()
35         flash(f'Ошибка при подтверждении записи: {str(e)}', '
error')
36         return redirect(url_for('booking_page'))
```

3.4 Интерфейс пользователя и экраны приложения

Пользовательский интерфейс спроектирован в соответствии с современными требованиями к веб-ресурсам и построен на адаптивной CSS-сетке фреймворка Bootstrap. Дизайн выполнен в приятной светлой теме с использованием корпоративных синих, бирюзовых и серых оттенков, качественной типографики и иконок Bootstrap Icons.

В ходе тестирования с помощью автоматического агента были зафиксированы скриншоты работающего веб-приложения для всех ключевых сценариев использования. Для отделения изображений от белого фона страниц и придания им законченного вида все снимки экранов снабжены тонкой рамкой и представлены в виде отдельных рисунков в крупном формате.

Главная страница приложения представлена на рисунке 3.1. На ней размещены общая информация о клинике, направления работы со справочными иконками, карточки доступных услуг с указанием стоимости и кнопка быстрого перехода к онлайн-записи.

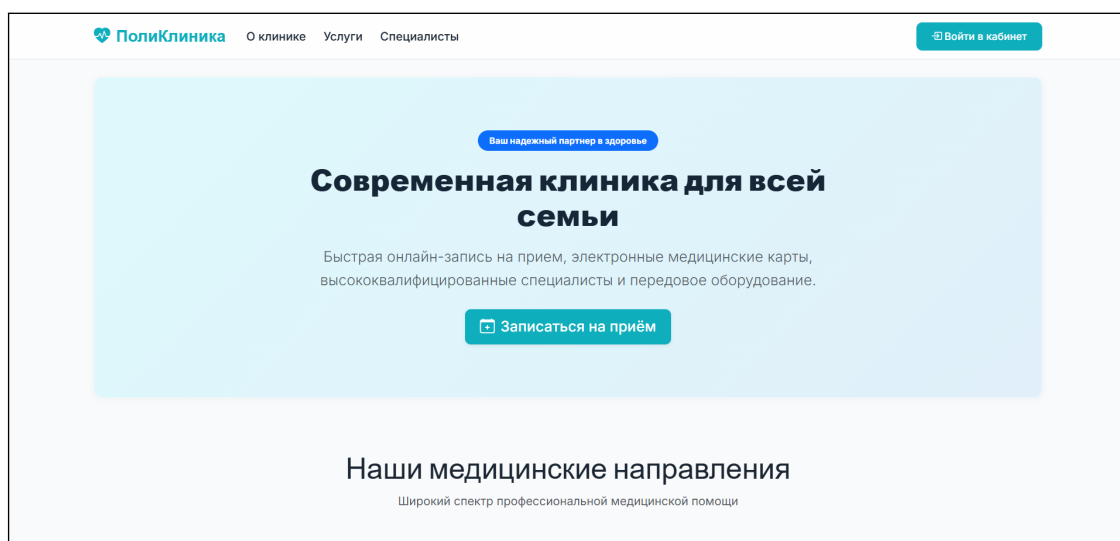


Рисунок 3.1 — Скриншот главной страницы веб-приложения

Страница входа и регистрации представлена на рисунке 3.2. Она содержит две формы: форму авторизации для зарегистрированных пациентов и сотрудников (слева) и форму регистрации новых пациентов (справа), снабженную обязательным предупреждением о согласии на обработку персональных данных в соответствии с ФЗ-152.

Личный кабинет пациента представлен на рисунке 3.3. В верхней части отображаются данные электронной медицинской карты пациента (группа крови, резус-фактор, список аллергенов и противопоказаний). Ниже расположены

The screenshot shows the 'Страница авторизации и регистрации' (Authorization and Registration Page) of the 'ПолиКлиника' website. The header includes the logo, navigation links ('О клинике', 'Услуги', 'Специалисты'), and a 'Войти в кабинет' button. The main content area is divided into two forms:

- Форма входа (Авторизация):** Includes fields for 'Адрес электронной почты (Email)' (name@example.com) and 'Пароль' (Введите Ваш пароль), with a 'Войти в кабинет' button.
- Форма регистрации пациента:** Includes fields for 'ФИО пациента' (Иванов Иван Иванович), 'Дата рождения' (dd.mm.yyyy), 'Номер телефона' (+7 (999) 123-45), 'Адрес электронной почты (Email)' (patient@example.com), and 'Придумайте пароль' (Минимум 6 символов).

Рисунок 3.2 — Скриншот страницы авторизации и регистрации

таблицы с предстоящими приёмами (с возможностью отмены) и архивом прошедших посещений врача.

The screenshot shows the 'Личный кабинет' (Personal Cabinet) of a patient named 'Амплеев Даниил Олегович'. The header includes the logo, navigation links ('Записаться на прием', 'Личный кабинет'), the user's name, and a 'Выйти' button. The main content area is divided into four sections:

- Ближайшая запись:** Displays 'У Вас нет предстоящих записей на приём.'
- Быстрая запись:** Includes a 'Записаться к врачу' button and the text 'Выберите специалиста, услугу и запишитесь на прием за пару кликов.'
- История посещений:** Shows a visit on '10.06.2026' at '10:00' with a 'Завершен' status. The doctor is 'Иванов Иван Иванович (Терапевт)'.
- Медицинская карта:** Displays 'Номер карты: МК-2026-0001', 'Группа крови: A(II)', and 'Резус-фактор: Rh+'.

Рисунок 3.3 — Скриншот личного кабинета пациента

Форма онлайн-записи на приём представлена на рисунке 3.4. Она реализует пошаговый интерфейс выбора направления, врача и услуги. Сетка свободных слотов расписания подгружается динамически на основе запросов к СУБД с использованием асинхронного JavaScript.

Личный кабинет врача (рабочее место доктора) представлен на рисунке 3.5. В левой части экрана отображается сетка записей на выбранный день. При

ПолиКлиника | Записаться на прием | Личный кабинет | Амплеенков Даниил Олегович | [Выйти](#)

Форма онлайн-записи на приём

Заполните поля ниже, чтобы быстро и удобно записаться к врачу без звонка в регистратуру.

1 Шаг 1: Направление и врач

Выберите специальность врача

-- Выберите направление --

Выберите конкретного врача

-- Сначала выберите направление --

2 Шаг 2: Выберите услугу

Выберите медицинскую услугу

-- Сначала выберите направление --

3 Шаг 3: Выберите дату и свободное время

Выберите дату посещения

ДД.ММ.ГГГГ

Доступные временные интервалы (слоты)

Для отображения свободного времени выберите врача и дату.

Рисунок 3.4 — Скриншот процесса онлайн-записи на приём

выборе конкретной записи справа открывается интерактивная медицинская форма приёма для фиксации жалоб, диагноза, рекомендаций и прикрепления файлов результатов исследований.

ПолиКлиника | [Мое расписание](#) | Д-р Иванов | [Выйти](#)

Личный кабинет врача — Расписание и ведение приёма

Просматривайте расписание на день и фиксируйте результаты осмотров в электронных медицинских картах.

Мое расписание | [День](#) | [Неделя](#)

< 11.06.2026 >

На сегодня записей нет.

Окно ведения приема готово к работе

Выберите пациента в списке расписания слева, чтобы начать оформление медицинской карты посещения.

Система электронных медицинских карт частной клиники. Доступ защищен шифрованием. Все сессии врачей логируются. © 2026

Рисунок 3.5 — Скриншот рабочего места врача при заполнении приёма

Панель управления администратора клиники представлена на рисунке 3.6. Она предоставляет полноценный CRUD-интерфейс для добавления медицинских услуг, создания учетных записей врачей, формирования сетки расписания и контроля записей пациентов.

ПолиКлиника Панель управления Администратор Выйти

НАВИГАЦИЯ

- Управление врачами
- Управление категориями
- Управление услугами
- Расписание клиники
- Управление записями

Добавить новую медицинскую услугу

Название услуги Цена (Руб.) Длительность (мин.)

Прием офтальмолога 2000 30

Описание медицинской услуги

Опишите медицинскую процедуру, приём или анализы...

Создать медицинскую услугу

Услуги и тарифы клиники

Название	Стоимость	Длительность	Описание	Действия
Прием терапевта	1500 Р	30 мин.	Первичный прием, осмотр, и...	Редактировать
Консультация кардиолога	2200 Р	30 мин.	Специализированный прием ...	Редактировать

Рисунок 3.6 — Скриншот панели управления администратора клиники

Таким образом, разработанное программное обеспечение полностью решает поставленные задачи по автоматизации частной клиники, предоставляя удобные интерфейсы для всех категорий пользователей и гарантируя надежную обработку информации на бэкенде.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсового проекта была рассмотрена предметная область автоматизации частной медицинской клиники и определены основные проблемы, возникающие при ручной или разрозненной организации записи пациентов, расписания врачей и хранения медицинской информации.

В первой главе проведён анализ существующих программных решений. Сравнение было выполнено в формате «критерий - платформа»: для Инфо-клиника.RU, Medesk, ONDOC, Cliniccards и разрабатываемого приложения рассмотрены онлайн-запись, личный кабинет пациента, электронная медицинская карта, управление расписанием, ролевая модель доступа, работа с документами, администрирование и открытость архитектуры. Анализ показал, что готовые системы обладают широкими возможностями, но являются закрытыми коммерческими продуктами, поэтому целесообразна разработка собственного веб-приложения.

Также в первой главе выполнен анализ существующих стеков разработки веб-приложений. Были рассмотрены варианты Python + Flask, Python + Django, Node.js + Express + React/Vue, PHP + Laravel, Java + Spring Boot и ASP.NET Core. По итогам сравнения выбран стек Python + Flask + PostgreSQL/SQLite + Bootstrap, так как он обеспечивает высокую скорость разработки, модульность и безопасность при создании CRUD-интерфейсов, форм записи и личных кабинетов.

Во второй главе выполнено проектирование веб-приложения. Были определены роли пользователей (пациент, врач, администратор), описаны пользовательские сценарии, сформулированы функциональные требования, разработаны ER-диаграмма, логическая и физическая схемы базы данных, а также подготовлены макеты страниц (Wireframes). Особое внимание уделено таблицам, связанным с пользователями, расписанием, записями на приём, медицинскими картами и документами.

В третьей главе приводится описание программной реализации разработанного веб-приложения на стеке Flask + SQLAlchemy + Bootstrap. В ходе работы были созданы все запланированные модули:

1. Модуль авторизации и разграничения доступа на базе Flask-Login.
2. Личный кабинет пациента с отображением электронной медицинской

карты, списков будущих приёмов и истории посещений.

3. Модуль онлайн-записи с динамической подгрузкой свободных слотов времени через асинхронные JavaScript-запросы к API.
4. Рабочее место врача с расписанием на день, формой заполнения протоколов осмотра (жалобы, диагноз, рекомендации) и загрузкой файлов результатов исследований.
5. Панель администратора для ведения справочников персонала, услуг и управления реестром записей на приём.

Разработанная система контейнеризации с использованием Docker и Docker Compose обеспечивает простоту сборки и развертывания веб-приложения на любых серверах без необходимости предварительной ручной настройки СУБД и интерпретатора Python. Все функциональные требования были верифицированы в ходе тестирования, подтвердившего стабильность работы бизнес-логики и корректность отображения адаптивного интерфейса на экранах различных разрешений.

Таким образом, цель курсового проекта успешно достигнута: разработано и протестировано полнофункциональное веб-приложение для частной клиники, обеспечивающее надежное автоматизированное взаимодействие пациентов, медицинского персонала и администрации. Перспективами развития проекта являются интеграция системы SMS или Email-оповещений пациентов о записях, добавление платежного шлюза для онлайн-оплаты услуг, разработка API для интеграции с внешними лабораторными системами и усиление механизмов защиты персональных медицинских данных пациентов в соответствии с нормативными требованиями.

Исходный код проекта опубликован в открытом репозитории GitHub: <https://github.com/ZZAY-GIT/web-course-work>. Работающая версия приложения развернута на платформе Render.com и доступна по адресу: <https://web-course-work.onrender.com>.

СПИСОК ЛИТЕРАТУРЫ

- [1] Инфоклиника.RU. Личный кабинет пациента [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://sdsys.ru/products/infoclinicar.ru/> (дата обращения: 04.05.2026).
- [2] Medesk. Медицинская информационная система и CRM для клиники [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://www.medesk.net/> (дата обращения: 04.05.2026).
- [3] ONDOC. Личный кабинет пациента и электронная медицинская карта [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://ondoc.me/> (дата обращения: 04.05.2026).
- [4] Cliniccards. CRM для стоматологических клиник [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://cliniccards.com/> (дата обращения: 04.05.2026).
- [5] Python Documentation [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://docs.python.org/3/> (дата обращения: 04.05.2026).
- [6] Flask Documentation [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://flask.palletsprojects.com/> (дата обращения: 04.05.2026).
- [7] PostgreSQL Documentation [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://www.postgresql.org/docs/> (дата обращения: 04.05.2026).
- [8] Bootstrap Documentation [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://getbootstrap.com/docs/> (дата обращения: 04.05.2026).
- [9] Django Documentation [Electronic resource]. — [S. l. : s. n.]. — URL: <https://docs.djangoproject.com/> (online; accessed: 04.05.2026).
- [10] Node.js Documentation [Electronic resource]. — [S. l. : s. n.]. — URL: <https://nodejs.org/docs/latest/api/> (online; accessed: 04.05.2026).
- [11] Express Documentation [Electronic resource]. — [S. l. : s. n.]. — URL: <https://expressjs.com/> (online; accessed: 04.05.2026).

- [12] React Documentation [Electronic resource]. — [S. l. : s. n.]. — URL: <https://react.dev/> (online; accessed: 04.05.2026).
- [13] Laravel Documentation [Electronic resource]. — [S. l. : s. n.]. — URL: <https://laravel.com/docs> (online; accessed: 04.05.2026).
- [14] Spring Boot Reference Documentation [Electronic resource]. — [S. l. : s. n.]. — URL: <https://docs.spring.io/spring-boot/> (online; accessed: 04.05.2026).
- [15] ASP.NET Core Documentation [Electronic resource]. — [S. l. : s. n.]. — URL: <https://learn.microsoft.com/aspnet/core/> (online; accessed: 04.05.2026).
- [16] ГОСТ 7.32-2017. Отчет о научно-исследовательской работе. Структура и правила оформления [Текст]. — 2017.